Verification Report

How the volatility surface tool was shown to be accurate

Kevin Williams | October 25, 2026

1 Summary

The tool turns option prices into implied volatilities and a surface. “Accurate” was tested at four levels, each against an answer known in advance or computed by an unrelated program. Everything below is marked by where it was observed.

Level	Result	Status
Pricing formula	Black-Scholes equation and put-call parity checked by MATLAB’s symbolic engine; reference prices computed independently in SciPy	Tests passed in MATLAB; reference comparison pending
Solver (price $\rightarrow$ volatility)	142 round-trip cases, worst error 3.1×10^{-7}	Observed in MATLAB
Surface vs a known surface	Interpolation matches a planted surface within test limits; holes stay blank, never zero	Observed in MATLAB
Real data (AAPL, 12 Oct 2026)	140 contracts, 8 expiries; agrees with the OCaml engine to 5×10^{-7}, and a third program (SciPy) agrees identically	Observed in MATLAB and Python
Historical stress test (March 2020)	Planted panic surface recovered; predicted worst miss about 1.2 volatility points	Written; MATLAB run pending

2 What “accurate” means here

Four separate things could be wrong, so each got its own test.

1. **The formula.** Is the pricing function itself correct?
2. **The solver.** Given a price, does it find the right volatility, or report that none exists?
3. **The surface.** Does interpolating between solved points reproduce the true shape, and stay honest where there is no data?
4. **The pipeline on real data.** Do cleaning, forward-price estimation and out-of-the-money selection behave on a messy chain?

3 Evidence, level by level

1. The formula

Two properties every correct option price must satisfy were tested: the Black-Scholes equation, and put-call parity (a call minus a put equals the stock minus the discounted strike). MATLAB’s symbolic engine builds the

price, derives vega, and evaluates both residuals at 50 random points; `test_vol_pipeline` requires them below 10^{-8} and passed. As an outside reference, a worked example (stock 100, strike 100, one year, 5% rate, 20% volatility) was computed in SciPy: call 10.450584, put 5.573526, vega 37.524035. `verify_demo` compares MATLAB against these numbers (pending its first run).

2. The solver

Prices were generated from known volatilities across 4 maturities, 5 strikes, 4 volatility levels, calls and puts, then solved backward. 142 cases were tested; the worst error was 3.1×10^{-7}. Prices that break no-arbitrage (a call below its intrinsic value, or worth more than the stock) correctly return "not a number". 18 cases were skipped on purpose: where vega is nearly zero the price carries no information about volatility, so no solver could recover it.

3. The surface

`test_surface` builds a surface from a formula whose true value is known everywhere, then queries 500 random points the interpolator was never given and requires interpolation error under 0.2 volatility points for volatility-space and 1 point for total-variance space (the default). It also requires that points outside the data hull come back blank, that cut-off corners show up as holes in the coverage figure, and that duplicate points and bad inputs are handled with clear errors. All passed.

4. Real data: three independent programs

The AAPL snapshot of 12 Oct 2026 gave 140 out-of-the-money contracts across 8 expiries. Every one was solved, none needed the bisection fallback.

Comparison	Median difference	Largest difference
MATLAB vs OCaml engine	2.6×10^{-7}	5.0×10^{-7}
SciPy (Brent solver) vs OCaml engine	2.55×10^{-7}	4.99×10^{-7}

The OCaml engine prints six decimals, so its output can be off by up to 5×10^{-7} from rounding alone. Both comparisons sit at exactly that ceiling. The SciPy solver converges to 10^{-12}, so the gap is OCaml's printing, not a disagreement about the answer.

5. Historical stress test (March 2020)

`verify_covid_crash` plants two surfaces, a calm one (19 Feb 2020) and a panic one calibrated to the VIX record close of 82.69 on 16 Mar 2020, prices options from them, runs the same pipeline as a live ticker and plots recovered against planted. It additionally checks that the story is read correctly: the panic term structure must be inverted and puts must cost more than calls. Free historical chains do not exist, so the surfaces are illustrative; the test targets the tool, not the market. A Python copy of the pipeline predicts a worst miss of about 1.2 volatility points on unseen panic points and 0.4 on calm ones, against an allowance of 2. The MATLAB run is pending.

4 Problems the verification caught

Testing was useful because it found real mistakes.

Symptom	Cause	Fix
Round trip off by 0.2 on some cases	Deep in-the-money, short-dated options have almost no vega, so volatility is not identifiable from the price	Test skips vega below 10^{-2}; tool uses out-of-the-money quotes only
"Outside the data must be blank" failed	MATLAB's `max(NaN,0)` returns 0, silently turning blank areas into zero volatility	Clamp only negative values; a test now guards coverage
5e-7 gap vs OCaml explained wrongly	Attributed to the solver's stopping tolerance	Corrected to OCaml's six-decimal printing after the SciPy check showed the same ceiling
Pre-open run produced a one-contract snapshot and overwrote the good one	Zero bids and asks before the open	Exporter refuses fewer than 20 rows or 2 expiries; prior-session mode added

5 How to reproduce

```
% MATLAB, from the matlab/ folder
test_vol_pipeline                       % formula, solver, pipeline
test_surface                            % surface and slices
verify_demo                             % one-page pass/fail report
verify_demo("snapshots/AAPL_20261012_1347.csv")   % adds real data vs OCaml
verify_covid_crash                      % historical stress test with figure

# terminal: independent solver on the same snapshot
python3 matlab/verify_independent.py snapshots/AAPL_20261012_1347.csv
```

6 Limits of this evidence

- **Agreement is not proof of fit.** MATLAB, OCaml and SciPy all solve the same equations with the same inputs (the exporter currently sets the dividend yield to zero). Matching each other shows the arithmetic is right, not that Black-Scholes describes the market perfectly.
- **One real snapshot.** Numerical cross-checks on real data used AAPL on one day. More tickers and dates would strengthen the claim.
- **The data itself.** Verification covers the computation. Yahoo quotes may be delayed or stale.
- **No arbitrage guarantee.** The surface is interpolated, not fitted with a no-arbitrage model (planned as a later project).
- **Pending runs.** `verify_demo` and `verify_covid_crash` were written after the earlier tests; their results are to be recorded below.

7 Results to record

Run	Date	Outcome	Notes
`verify_demo`			
`verify_demo` + snapshot			
`verify_covid_crash`			
`verify_independent.py`	25 Oct 2026	PASS, max 4.99×10^{-7}	Python, AAPL 12 Oct